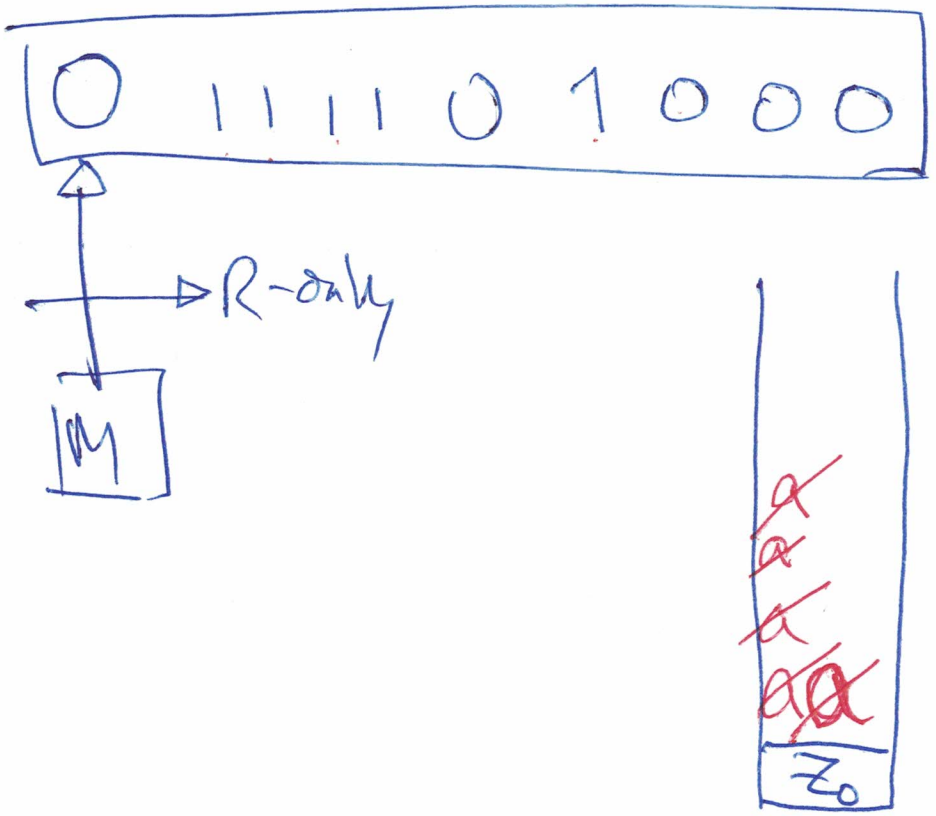
Cpts 317. 10/28.

Construct a PDA M to accept

$$\mathcal{L} = \{ w \in \{0,1\}^* : \#_0(w) = \#_1(w) \}.$$

English Construction:



two modes:

mode1: see 0 → push

see 1 → pop

mode2: see 0 → pop

see 1 → push.

when stack is empty, we guess a mode to go.

Explicit Construction:

$$\delta(q_0, \Lambda, Z_0) = \{(q_1, Z_0), (q_2, Z_0)\}$$

// guess one of the
// two modes to go

$$\delta(q_1, 1, a) = \{(q_1, \Lambda)\}$$

$$\delta(q_1, 0, *) = \{(q_1, a*)\}, \quad * \in \{Z_0, a\}.$$

$$\delta(q_1, \Lambda, Z_0) = \{(q_0, Z_0)\}$$ // go back to
// q_0 to guess
// next move when
// stack is empty

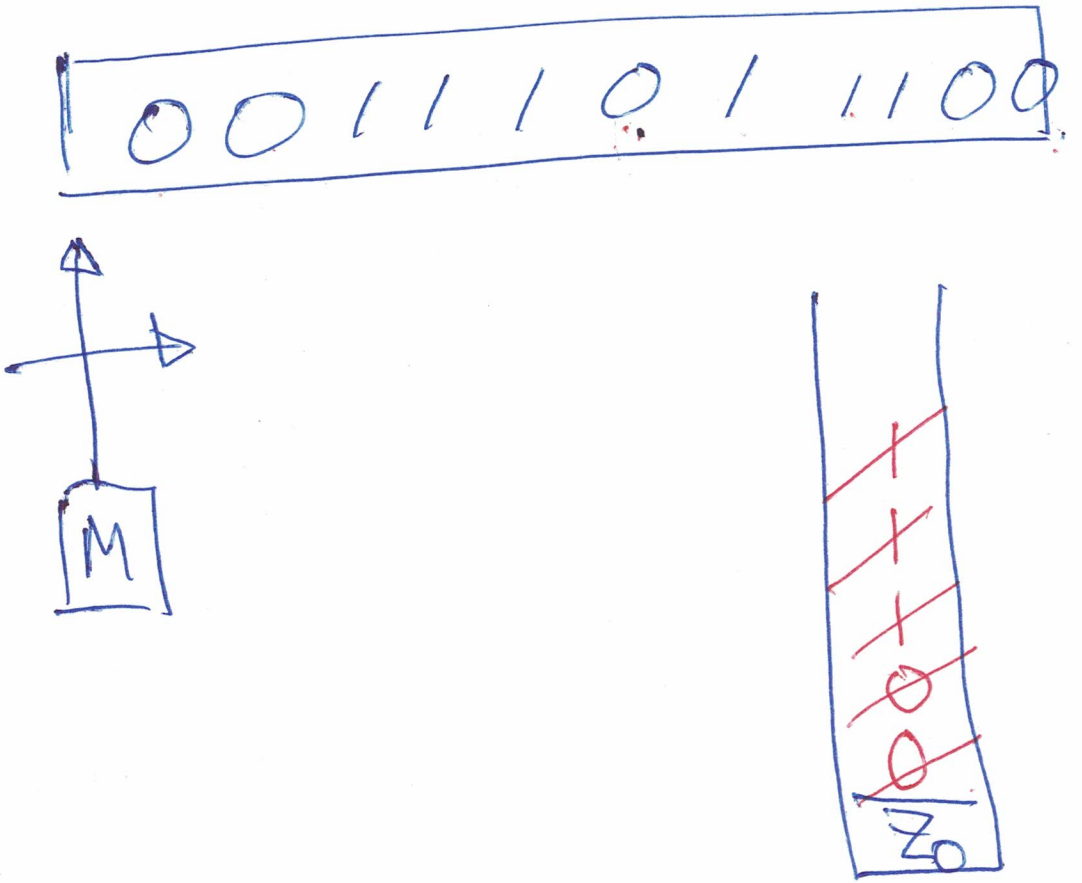
$\delta(q_2, 1, *) = \{(q_2, a*)\}, \quad * \in \{Z_0, a\}.$

$\delta(q_2, 0, a) = \{(q_2, \Lambda)\}$

$\delta(q_2, \Lambda, Z_0) = \{(q_0, Z_0)\}$ // go back to
// q_0 to guess next
// mode when stack
is empty

// q_0 is also accepting state.

Construct a PDA accepting $\mathcal{L} = \{ w \in \{0,1\}^* : w = w^r \}$.



Construction in English.

M runs on input word with empty stack as follows. For each symbol M reads, it pushes it onto the stack. Some moments later, M guesses now it reads the middle. At this moment, M also guess the len of the input word is even or odd. If it is odd, M skip the ~~next input~~ current input symbol. Then, M continues to read the rest of the input and, at same time, pop & compare with the stack top symbol. M says yes at the end of input if stack is empty.

Explicit Construction:

$$\delta(q_0, *_1, *_2) = \{(q_0, *_1 *_2)\},$$

$$*_1 \in \{0, 1\}$$

$$*_2 \in \{0, 1, Z_0\}.$$

// Keep push input symbols
// onto the stack!

$$\delta(q_0, \Lambda, *) = \{(q_{even}, *), (q_{odd}, *)\},$$

$$* \in \{0, 1, Z_0\},$$

// you guess the len to be
// even or odd.

$\delta(q_{odd}, *_1, *_2) = \{(q_{even}, *_2)\}$

// skip the middle

$\delta(q_{even}, *, *) = \{(q_{even}, \Lambda)\}, \; * \in \{0,1\}.$

// read & compare with
// the top of stack.

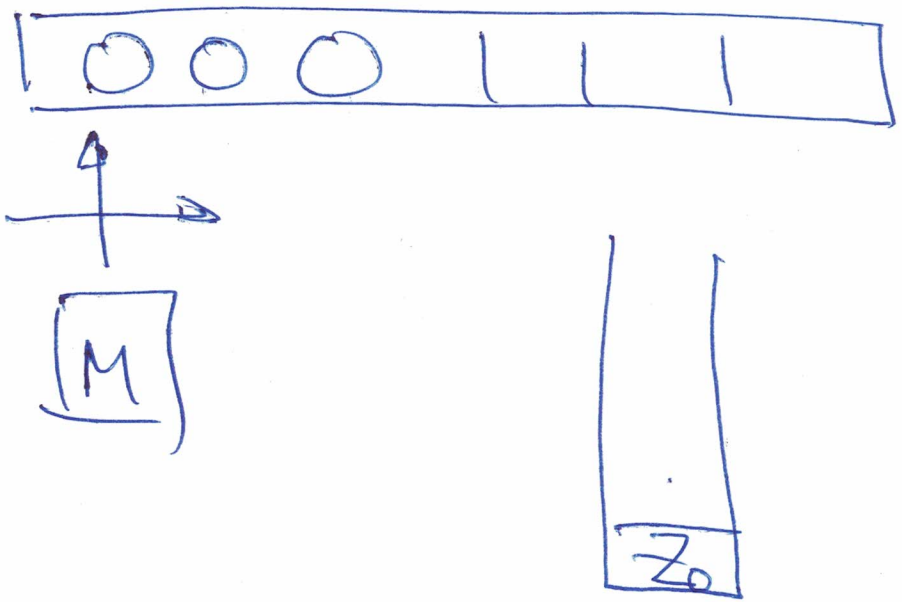
$\delta(q_{even}, \Lambda, Z_0) = \{(q_2, Z_0)\}$ // q_2 is
// accepting

$\delta(q_0, \Lambda, Z_0) = \{(q_2, Z_0)\}$ /// $\Lambda \in \mathcal{L}$.

DPDA: deterministic PDA.

From one state, one input symbol, one stack top,
There is at most one next move.
// no guess in DPDA.

Ex: $\mathcal{L} = \{0^n 1^n : n \geq 1\}$ can be accepted by DPDA.



Recall: NFA $\xrightarrow{\text{subset construction}}$ DFA

(nondet. FA) (det. FA)

n states 2^n states.

PDA $\longrightarrow$ DPDA

M M'

Such that $L(M) = L(M')$? NO.

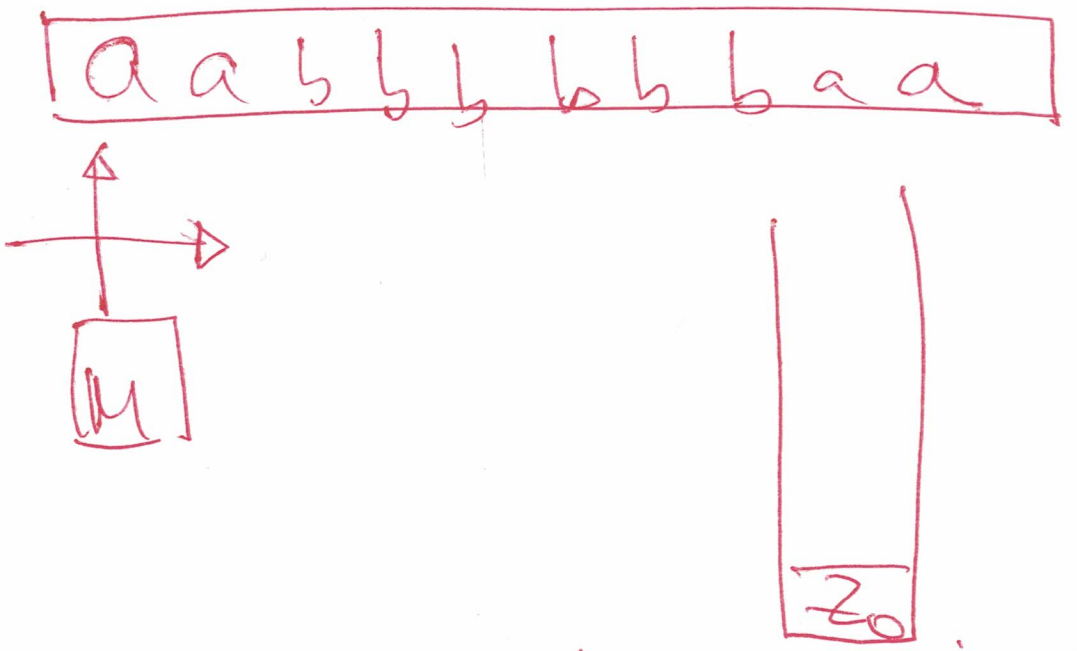
Conclusion, DPDA is strictly weaker than PDA.

To prove it, we only show that there is a language accepted by PDA, but not by DPDA.

Thm. $\mathcal{L} = \{w : w = w^r\}$ on $\{a, b\}$ is accepted by PDA. But $\mathcal{L}$ can not be accepted by DPDA.

Proof. (Wrong Proof —— don't do it).

(Bad proof).

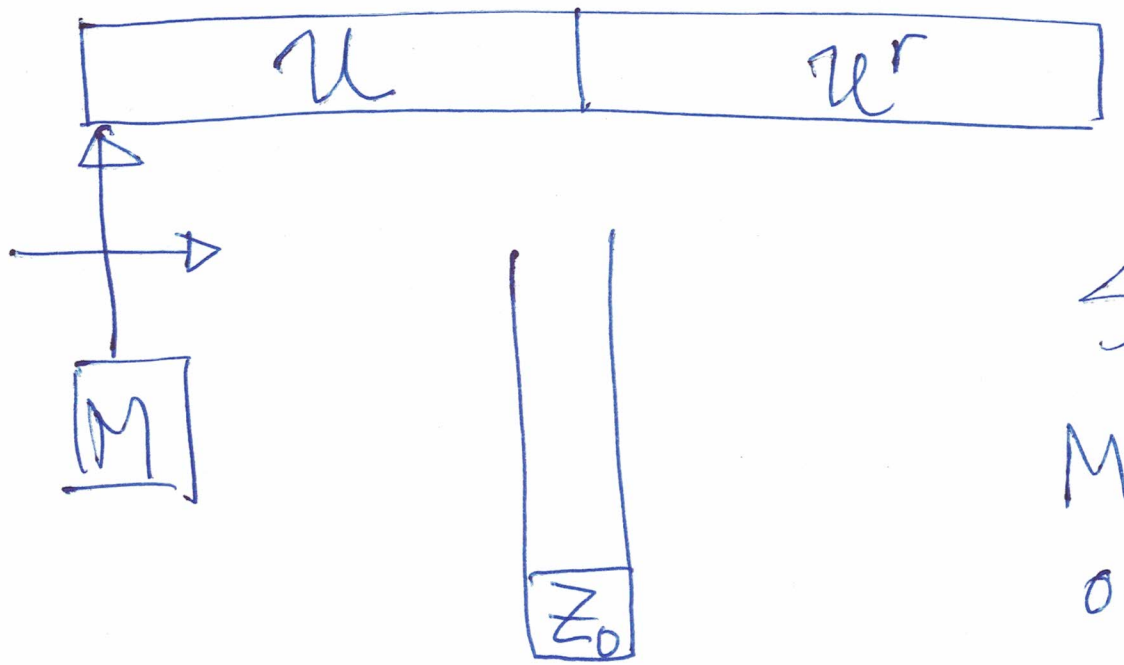


When M reads the input, it has to decide the middle position in order to check $w = w^r$. But the middle can't be decides without reading the entire input. Hence, when M is in the middle, it has to guess. ▨

(good proof —— proof by contradiction).

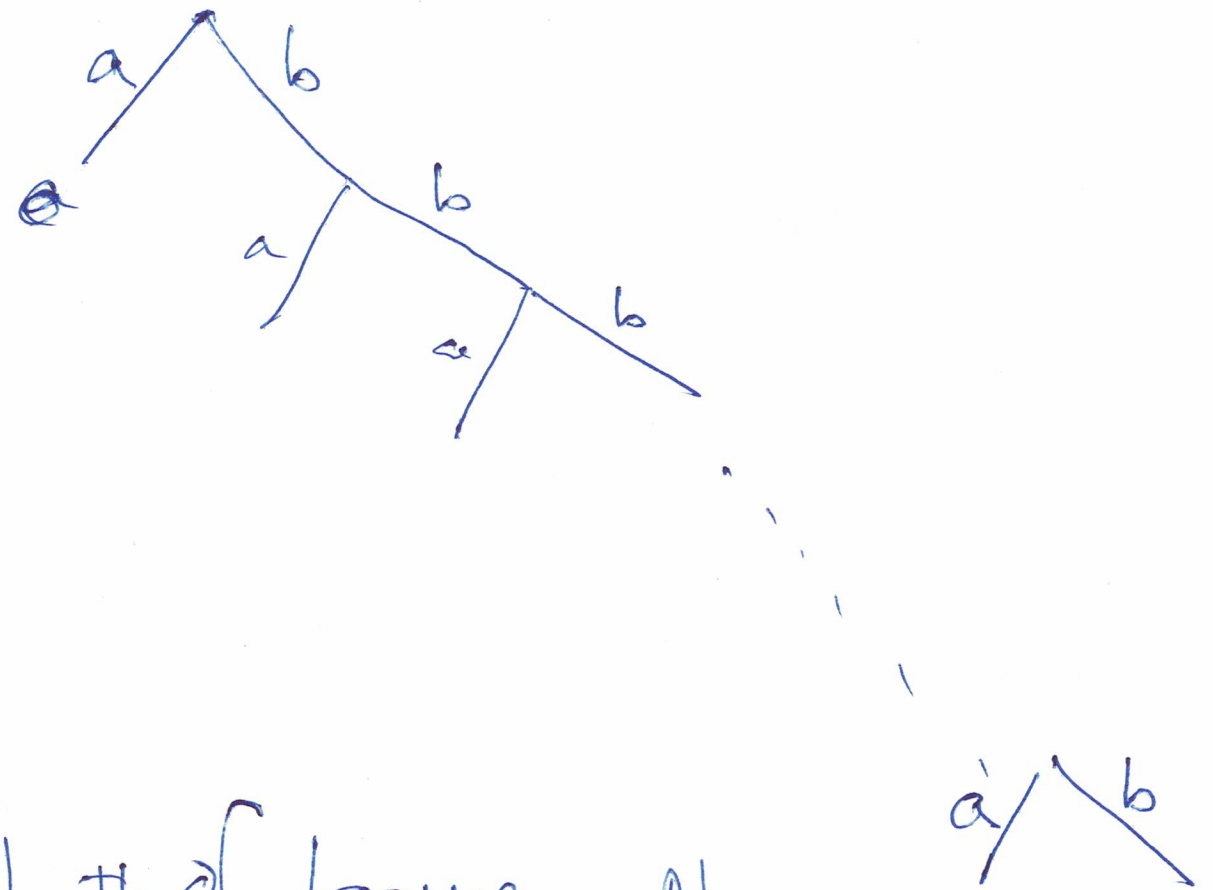
Assume that $\mathcal{L}$ can be accepted by a DPDA M.

Ob 1. On an arbitrary input word u, M will never crash. why?



Since $uu^r \in \mathcal{L}$, M has to say yes on uu^r.

Ob2: Let $N_0 = |Q| \times |P|$. I build a prefix-tree:



total # of leaves $= N_0 + 1$